



TRIBHUVAN UNIVERSITY
INSTITUTE OF SCIENCE AND TECHNOLOGY



HIMALAYA COLLEGE OF ENGINEERING
CHYASAL, LALITPUR



Lab Report No:- 8

Title:- Implementation of knn algorithm.

Submitted by:-

Name:- Surajan Shrestha.

Roll No:- 38

Submitted To:- Rubas mali

Department Of CSIT

Checked by:-

Date of submission:- 2081/06/05

TITLE: Implementation of KNN Algorithm.

OBJECTIVE:

→ The primary objective of this lab is to implement the K-nearest Neighbors (KNN) algorithm in Python using the Jupyter Notebook environment. This implementation aims to understand the basic working principle of KNN, evaluate its performance on a dataset, and visualize the results. The Iris dataset will be used to classify different species of flowers based on their features.

DEFINITION:

KNN is a non-parametric, instance-based learning algorithm used for classification and regression tasks. It is one of the simplest machine learning algorithms and is based on the concept of proximity or distance between data points.

Key concepts:

Non-parametric Model: KNN does not make any assumptions about the underlying data distribution.

Instance-Based Learning: It memorizes the training dataset instead of building an explicit model.

Distance Metric: KNN uses distance metrics such as Euclidean, Manhattan, or Minkowski to determine the closeness of data points.

How KNN works:

- **Choose K:** Select the number of nearest neighbors.
- **Distance Calculation:** Calculate the distance between

the test data point and all the training data points using a chosen distance metric (commonly Euclidean distance).

- Neighbor selection : Identify the k nearest neighbors based on the shortest distance.
- Voting mechanism : for classification, the class of the majority of the neighbors is assigned to the test point. for regression, the mean of neighbors is considered.
- Prediction : The label or value of the test point is determined based on k neighbors.

Applications of KNN :

- Classification tasks such as handwritten digit recognition, image classification & pattern recognition.
- Regression tasks involving prediction of numerical values.
- Recommendation systems and anomaly detection.

Advantages :

- Simple to understand and implement.
- Effective with a small amount of training data.
- No need for training ; it is a lazy learner.

Disadvantages :

- Computationally expensive with large datasets.
- Sensitive to the choice of k and distance metric.
- Performs poorly with imbalanced datasets.

OBSERVATIONS :

→ The following steps detail the implementation of KNN algorithm using two datasets:

1> Data loading and preprocessing:

• for classification.csv.csv:

code:

```
# load classification.csv.csv dataset
```

```
df1 = pd.read_csv('classification.csv.csv')
```

```
print(df1.head())
```

```
# split the data into features and target
```

```
x1 = df1[['feature_1', 'feature_2']]
```

```
y1 = df1['class']
```

```
# split the dataset into training and testing sets
```

```
X_train1, X_test1, y_train1, y_test1 = train_test_split
```

```
(x1, y1, test_size=0.3, random_state=42)
```

```
# feature scaling
```

```
Scaler1 = StandardScaler()
```

```
X_train1 = Scaler1.fit_transform(X_train1)
```

```
X_test1 = Scaler1.transform(X_test1)
```

• for practice.csv:

code:

```
# load practice.csv dataset
```

```
df2 = pd.read_csv('practice.csv')
```

```
print(df2.head())
```

```
# split the data into features and target
```

```
x2 = df2[['x1', 'x2']]
```

```
y2 = df2['class']
```



```
# Split the dataset into training and testing sets
X_train2, X_test2, Y_train2, Y_test2 = train_test_split(X2, Y2, test_size = 0.3, random_state = 42)
```

```
# feature scaling
```

```
scaler2 = StandardScaler()
```

```
X_train2 = scaler2.fit_transform(X_train2)
```

```
X_train X_test2 = scaler2.transform(X_test2)
```

2> Training the KNN model:

- for classification.csv:

```
# Initialize and train KNN classifier
```

```
knn1 = KNeighborsClassifier(n_neighbors=3)
```

```
knn1.fit(X_train1, Y_train)
```

- for practice.csv:

```
# Initialize and train KNN classifier
```

```
knn2 = KNeighborsClassifier(n_neighbors=3)
```

```
knn2.fit(X_train2, Y_train2)
```

3> Prediction and Evaluation:

- performance evaluation is done separately for each dataset using accuracy scores and confusion matrices.

- for classification.csv.csv:

```
# predict and evaluate on classification.csv.csv
```

```
Y_pred1 = knn1.predict(X_test1)
```

```
accuracy1 = accuracy_score(Y_test1, Y_pred1)
```

```
cm1 = confusion_matrix(Y_test1, Y_pred1)
```

```
print(f'Accuracy on classification.csv.csv % {accuracy1}')
```

```
print(f'Confusion matrix :\n', cm1)
```

for pratice.csv:

```
# predict and evaluate on pratice.csv
y_pred2 = knn2.predict(x_test2)
accuracy2 = accuracy_score(y_test2, y_pred2)
cm2 = Confusion_matrix(y_test2, y_pred2)
print(f'Accuracy on pratice.csv : {accuracy2 * 100}%')
print('Confusion matrix :\n', cm2)
```

OUTPUT :

Accuracy on classification.csv : 97.78%

Confusion matrix :

$\begin{bmatrix} 1470 & 5 \\ 20 & 1505 \end{bmatrix}$

Accuracy on pratice.csv : 96.45%

Confusion matrix :

$\begin{bmatrix} 1452 & 28 \\ 30 & 1490 \end{bmatrix}$

4, Visualizing the Decision Boundaries:

- visualization for each dataset helps to understand how the KNN model classifies different classes based on the provided features.

- for classification.csv:

```
# visualization for classification.csv
```

```
x_min1, x_max1 = x1['feature_1'].min() - 1, x1['feature_1']
```

```
max1() + 1
```



```

x_min1, x_max1 = x1 ['feature_2'].min() - 1, x1 ['feature_2'].max() + 1
y_min1, y_max1 = np.arange(x_min1, x_max1, 0.1), np.arange(y_min1, y_max1, 0.1)
z1 = knn1.predict(scaler1.transform(np.c_[xx1.ravel(), yy1.ravel()]))
z1 = z1.reshape(xx1.shape)
plt.contourf(xx1, yy1, z1, alpha=0.3)
plt.xlabel('feature_1')
plt.ylabel('feature_2')
plt.title('kNN Decision Boundary for classification.csv')
plt.show()

```

• for practice.csv:

visualization for practice.csv

```

x_min2, x_max2 = x2 ['x1'].min() - 1, x2 ['x1'].max() + 1
y_min2, y_max2 = x2 ['x@'].min() - 1, x2 ['x@'].max() + 1
xx2, yy2 = np.meshgrid(np.arange(x_min2, x_max2, 0.1), np.arange(y_min2, y_max2, 0.1))
z2 = knn2.predict(scaler2.transform(np.c_[xx2.ravel(), yy2.ravel()]))
z2 = z2.reshape(xx2.shape)
plt.contourf(xx2, yy2, z2, alpha=0.3)
plt.scatter(x2 ['x1'], x2 ['x@'], c=y2, edgecolor='k', marker='o')
plt.xlabel('x1')
plt.ylabel('x@')
plt.title('kNN Decision Boundary for practice.csv')
plt.show()

```

DISCUSSION :

→ The performance of KNN algorithm is significantly influenced by the choice of k , feature scaling and the selected distance metric. A smaller k value can make the model overly sensitive to noise, while a larger k smooths boundaries, enhancing robustness but potentially reducing accuracy. Feature scaling using `StandardScaler` is crucial as it balances the influence of features, improving the model's accuracy. In this implementation, the Euclidean distance was used, but other metrics like Manhattan or Minkowski could be explored depending on the dataset.

CONCLUSION :

→ The implementation of KNN on two different database demonstrates the versatility of algorithm in classification tasks. The accuracy results affirm KNN's effectiveness, though future work could explore the impact of tuning parameters like k and different distance metrics to optimize performance.